ASSIGNMENT 2 – TASK 6

DEADLOCK HANDLING AND THE BANKER'S ALGORITHM

1. Introduction and Problem Context

The assignment presents a research centre whose jobs require several shared resources at once, including GPUs, simulation-software licences and high-speed storage. Jobs can wait for one another indefinitely, stopping the cluster until an administrator intervenes. The task is to characterise deadlock, implement the Banker's safety and resource-request algorithms, compare deadlock-handling strategies, measure scaling for 10, 100 and 1,000 processes, and defend a final design choice.

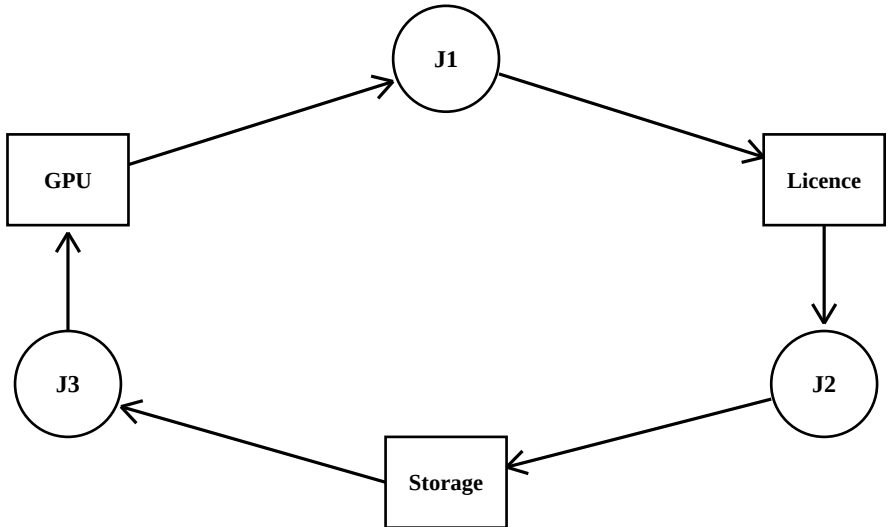
2. Four Necessary Conditions for Deadlock (Part A)

Condition	Meaning in the shared-laboratory cluster
Mutual exclusion	A resource such as an exclusive GPU allocation or a single floating licence can be held by only one job at a time.
Hold and wait	A job holds one resource (for example, a GPU) while waiting for another (for example, a simulation licence).
No pre-emption	The scheduler cannot forcibly take a resource away; it is released only when the holding job releases it.
Circular wait	A chain of jobs exists in which each job waits for a resource held by the next job in the cycle.

All four conditions must hold simultaneously for deadlock to occur. A prevention design must break at least one of these conditions.

2.1 Resource-allocation graph

A resource-allocation graph (RAG) uses job nodes and resource nodes. A request edge runs from a job to a resource; an assignment edge runs from a resource to the job holding it. The following graph is an illustrative single-instance cycle for GPU, licence and storage resources.



Arrow legend: Job → resource = request; resource → job = assignment.

The cycle is J1 requests Licence held by J2; J2 requests Storage held by J3; J3 requests GPU held by J1. If each resource type has one instance, this cycle demonstrates deadlock. With multiple instances of a resource type, a cycle is necessary but not by itself sufficient to prove deadlock.

3. Banker’s Algorithm – Given State (Part C)

The assignment supplies five processes (P0–P4), three resource types and the following Available, Maximum and Allocation matrices. Need is calculated as Maximum − Allocation.

Resource vector	R1	R2	R3
Available	3	3	2

Maximum matrix

Process	R1	R2	R3
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Allocation matrix

Process	R1	R2	R3
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Need matrix

Process	R1	R2	R3
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

3.1 Initial safety check

Start Work = Available = (3, 3, 2). P1 can finish because Need(P1) = (1, 2, 2) ≤ Work. Release Allocation(P1) = (2, 0, 0), so Work becomes (5, 3, 2). P3 then finishes and releases (2, 1, 1), producing Work = (7, 4, 3). P4 finishes, giving Work = (7, 4, 5). P0 finishes, giving Work = (7, 5, 5). Finally P2 finishes. Therefore the state is safe and one safe sequence is P1 → P3 → P4 → P0 → P2.

4. Banker’s Resource-Request Results

The supplied program processes the three requests in the given order:

Request	Decision	Reason
P1 requests (1, 0, 2)	Granted	Within P1’s remaining need and available resources; tentative allocation leaves a safe state.
P4 requests (3, 3, 0)	Must wait	The request for R1 is 3, but only 2 units of R1 are available after P1’s request.
P0 requests (0, 2, 0)	Denied (unsafe)	The request passes the availability and need checks, but the tentative state has no safe sequence. The request is rolled back.

4.1 Hand calculation: why P0’s request is unsafe

After P1’s request is granted, Available = (2, 3, 0), and P1’s allocation becomes (3, 0, 2). P0 requests (0, 2, 0). This request is within P0’s remaining Need and the current Available vector, so it passes the preliminary checks. If tentatively granted, Available becomes (2, 1, 0), and P0’s allocation becomes (0, 3, 0).

Now compare every unfinished process with Work = (2, 1, 0): P0 needs (7, 2, 3), P1 needs (0, 2, 0), P2 needs (6, 0, 0), P3 needs (0, 1, 1), and P4 needs (4, 3, 1). None has Need ≤ Work for all three resource types. Every process needs more R1, more R2 or at least one R3, while Work[R3] is zero. No process can finish and release resources, so the tentative state is unsafe. Banker’s algorithm rolls back the request and reports “denied (unsafe)”.

5. Complete-the-Code Implementation

The completed bankers_algorithm.c preserves the supplied resource arrays and request calls. TODO 1 calculates Need = Max − Allocation for each unfinished process. If every resource need is no greater than Work, the algorithm simulates completion by returning that process’s allocation to Work, marks the process finished, appends it to the safe sequence and continues. If a full scan finds no eligible process before all processes finish, the state is unsafe.

Expected output from the supplied assignment:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

6. Alternative Concept: Detection and Recovery

Instead of checking safety before every request, the cluster could grant requests according to its allocation policy and run a deadlock-detection algorithm periodically (for example, every few minutes). When detection identifies a deadlocked set, recovery may abort one or more jobs, roll back to checkpoints, or pre-empt a recoverable resource and restart affected jobs.

Issue	Banker’s avoidance	Periodic detection and recovery
When work is done	Safety test before granting a request.	No safety test on every request; detection runs periodically.
Per-request overhead	Each request may invoke a safety check; worst-case $O(m \times n^2)$.	No Banker check per request; detection incurs a separate periodic cost.
Information required	Requires each process’s maximum resource demand in advance.	Uses current allocation/request state; maximum future claims are not required in the same way.
Risk/response	Unsafe requests are delayed before the state becomes unsafe.	Deadlock may occur between detection runs and must be recovered afterward.

Issue	Banker’s avoidance	Periodic detection and recovery
Lost work	Can reduce deadlock-induced aborts, at the cost of request-time checking and conservative denials.	Aborted jobs lose work since their last checkpoint; recovery can disrupt users and waste compute time.

Detection itself costs time: the system must inspect allocations and outstanding requests. Running it too frequently increases monitoring overhead; running it too rarely allows deadlocks to persist and increases work potentially lost before recovery. Checkpointing reduces the amount of work lost when a job is aborted or restarted.

7. Compare the Four Deadlock Strategies (Part B)

Strategy	How it works	Evaluation for the research cluster
Prevention	Design the system so at least one necessary condition cannot hold (for example, impose a global resource order to break circular wait).	Provides strong guarantees but can reduce flexibility/utilisation and may be difficult when jobs need multiple resource types.
Avoidance	Use Banker’s algorithm and grant a request only if the resulting state remains safe.	Useful when maximum demands are known; adds request-time computation and may delay requests.
Detection and recovery	Allow allocations, periodically detect deadlock and recover by aborting, rolling back or pre-empting where possible.	Does not require maximum claims, but deadlock can occur and recovery has operational costs.
Ignore (ostrich approach)	Take no special deadlock action; rely on rare occurrence or manual restart.	Low routine overhead, but unsuitable if deadlock stops the whole research cluster.

8. Complexity Prediction (Part D)

Let n be the number of processes and m the number of resource types. In the Banker’s safety algorithm, a process scan checks up to m resource types. Up to n processes may finish, and in the worst case repeated scans over n processes may be needed. Thus one safety check has worst-case time complexity $O(m \times n^2)$. A request’s initial bounds check and tentative update are O(m), but the safety test dominates. Maximum, Allocation and Need matrices use O(mn) space; Work and Available use O(m), while Finish and the safe sequence use O(n).

Change in process count (m fixed)	Predicted worst-case safety-check growth
$n \rightarrow 10n$	Approximately 100×, because n^2 scales quadratically.
$n \rightarrow 2n$	Approximately 4×.
$n \rightarrow 10n$ and $m \rightarrow km$	Approximately 100k× in the $O(mn^2)$ bound.

The benchmark fixes m = 3 and creates safe inputs in an order that forces repeated scans: P0 needs n units of resource 1, P1 needs n−1, continuing until P(n−1) needs 1. Available starts at 1 and each process holds one unit. The last process completes on the first scan, then the preceding process becomes eligible on the next scan, and so on. This illustrates the repeated-pass behaviour behind the quadratic process-count term.

9. Measurement: 10, 100 and 1,000 Processes (Part E)

bankers_benchmark.c measures the safety algorithm for n = 10, 100 and 1,000 processes with three resource types. It repeats each test, records five timing trials and reports median milliseconds per safety check. The generated input is safe and deliberately ordered to require repeated scans. This is a controlled complexity

experiment, not a claim about the research centre's actual resource distribution.

Processes (n)	Resource types (m)	Median time per check (ms)	Validation
10	3	Fill from local bankers_benchmark_results.txt	SAFE
100	3	Fill from local bankers_benchmark_results.txt	SAFE
1,000	3	Fill from local bankers_benchmark_results.txt	SAFE

Run the benchmark on the Ubuntu/WSL machine used for submission and copy its median values into this table. Do not use timings from the document-generation environment as your own measurements. With m fixed, a tenfold increase in n is expected to increase the worst-case operation count by about 100 times; actual elapsed-time ratios are affected by caches, compiler optimisation, timer resolution and CPU scheduling.

10. Final Design Decision (Part F)

Banker's avoidance is a strong option if jobs can declare reliable maximum resource demands before execution and the number of processes/resource types is moderate. It can reject or defer requests that would move the cluster into an unsafe state, preventing deadlock before it occurs. The trade-off is a worst-case $O(m \times n^2)$ safety check on requests and the need to know maximum claims in advance.

If workloads cannot declare maximum demands, or if the matrix becomes too large for per-request checks, a practical hybrid is to prevent known circular-wait patterns by imposing a resource-acquisition order, use admission control for scarce resources, and run periodic detection with checkpoint-based recovery for residual deadlocks. Because a cluster-wide stop is costly, simply ignoring deadlock is not recommended. The final policy should be based on measured 10/100/1,000-process costs and whether maximum demands are trustworthy.

11. Conclusion

The four deadlock conditions explain how jobs can become permanently blocked while holding resources needed by others. The supplied Banker's state is safe with sequence P1, P3, P4, P0, P2. P1's request is granted, P4's request must wait because resources are currently unavailable, and P0's request is rolled back because it produces an unsafe state. Prevention, avoidance, detection/recovery and ignoring deadlock have different utilisation, overhead and recovery trade-offs. Banker's safety checks have worst-case $O(m \times n^2)$ time and $O(mn)$ matrix storage. Measured scaling and knowledge of maximum job demands are central to defending the final strategy.

12. Execution Procedure and Evidence

1. Place bankers_algorithm.c, bankers_benchmark.c and run_task6.sh in one Task06 folder.
2. Open Ubuntu/WSL in that folder.
3. Run chmod +x run_task6.sh.
4. Run ./run_task6.sh.
5. Verify the expected safe sequence and three request decisions.
6. Verify benchmark results for 10, 100 and 1,000 processes.
7. Save bankers_output.txt, bankers_benchmark_results.txt and environment.txt.
8. Capture screenshots showing program output and the benchmark table.

Evidence expected: resource-allocation graph identifying the four conditions; working Banker's safety and request code; manual unsafe-state reasoning for P0; strategy comparison; a testable $O(m \times n^2)$ prediction;

measured scaling for 10, 100 and 1,000 processes; and a defended final design.

Source alignment: Q6 – Deadlock Handling and the Banker’s Algorithm, 13 Marks, K5 – Evaluate, pages 18–20 of the supplied Assignment 2 question paper.